

Pandas Master Notes (Beginner → Intermediate)

1. Overview

Pandas is the most popular Python library for:

- Data analysis
- Data manipulation
- Data cleaning
- CSV/Excel handling
- Data preprocessing
- Machine learning preparation

Pandas is built on top of:

```
NumPy
```

which means:

```
Pandas is fast and vectorized.
```

2. Setup & Installation

Install Pandas

```
pip install pandas
```

Import Pandas

```
import pandas as pd
```

Why `pd`?

`pd` is the standard alias used worldwide.

Instead of writing:

```
pandas.read_csv()
```

we write:

```
pd.read_csv()
```

This makes code shorter and cleaner.

3. What is Pandas?

Pandas is an open-source Python library designed for:

- Structured data
- Tabular data
- CSV files
- Excel files
- SQL tables
- APIs
- JSON data

Pandas makes data analysis:

- Fast
 - Flexible
 - Easy
 - Intuitive
-

4. Core Data Structures in Pandas

Pandas mainly has 2 important data structures.

Series

A Series is:

A 1D labeled array.

Think of it like:

A single column in Excel.

Example:

```
import pandas as pd

s = pd.Series([10, 20, 30])
print(s)
```

Output:

```
0    10
1    20
2    30
dtype: int64
```

DataFrame

A DataFrame is:

A 2D tabular data structure.

Think of it like:

Excel spreadsheet or SQL table.

A DataFrame has:

- Rows
 - Columns
 - Labels
 - Different datatypes
-

Key Features of DataFrame

- Label-based indexing
 - Row and column operations
 - Fast vectorized operations
 - Supports mixed datatypes
 - Easy filtering and cleaning
-

5. Creating a DataFrame

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'Country': ['USA', 'Canada', 'UK']
}

df = pd.DataFrame(data)
print(df)
```

Output:

	Name	Age	Country
0	Alice	25	USA
1	Bob	30	Canada
2	Charlie	35	UK

Why DataFrames are Powerful?

Because they combine:

- Database-like operations
- Spreadsheet-like viewing
- Python programmability

all together.

6. Using Jupyter Notebook

Jupyter Notebook is commonly used with Pandas.

Why?

Because:

- Tables display beautifully
- Easier visualization
- Interactive coding
- Perfect for data analysis

Install:

```
pip install notebook
```

Run:

```
jupyter notebook
```

7. Loading CSV Files

One of the most important Pandas tasks.

Load CSV

```
import pandas as pd

df = pd.read_csv('orders.csv')
print(df)
```

Why `read_csv()`?

CSV files are:

The most common data format in data science.

Pandas converts CSV directly into a DataFrame.

Load Excel File

```
df = pd.read_excel('data.xlsx')
```

Load from Dictionary

```
df = pd.DataFrame(data)
```

Example Orders Dataset

```
import pandas as pd

orders = {
    'OrderID': [1001,1002,1003,1004,1005],
    'CustomerName': ['John Smith','Sarah Lee','Ali Khan','Alice Wong','Carlos Mendez'],
    'Product': ['Laptop','Headphones','Office Chair','Desk Lamp','Keyboard'],
    'Category': ['Electronics','Electronics','Furniture','Furniture','Electronics'],
    'Quantity': [1,2,1,3,2],
    'Price': [1200.0,150.0,300.0,45.0,80.0]
}

df = pd.DataFrame(orders)
print(df)
```

Output:

	OrderID	CustomerName	Product	Category	Quantity	Price
0	1001	John Smith	Laptop	Electronics	1	1200.0
1	1002	Sarah Lee	Headphones	Electronics	2	150.0

2	1003	Ali Khan	Office Chair	Furniture	1	300.0
3	1004	Alice Wong	Desk Lamp	Furniture	3	45.0
4	1005	Carlos Mendez	Keyboard	Electronics	2	80.0

8. Exploring a DataFrame

Before analyzing data, we first explore it.

df.head()

```
print(df.head())
```

Output:

```
First 5 rows of dataset
```

Why use `head()`?

Useful for:

- Previewing data
 - Checking columns
 - Quickly inspecting large datasets
-

df.tail()

```
print(df.tail())
```

Shows last 5 rows.

Why use `tail()`?

Useful for:

- Checking ending records
 - Verifying imported data
 - Inspecting latest entries
-

`df.info()`

```
print(df.info())
```

Example Output:

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 5 entries, 0 to 4  
Data columns (total 6 columns):
```

Why use `info()`?

Very important for:

- Checking missing values
 - Understanding datatypes
 - Dataset size
 - Memory usage
-

`df.describe()`

```
print(df.describe())
```

Why use `describe()`?

Provides statistical summary:

- Mean
- Standard deviation
- Min/max
- Quartiles

Useful in:

- Data analysis
 - ML preprocessing
-

`df.columns`

```
print(df.columns)
```

Output:

```
Index(['OrderID', 'CustomerName', 'Product'], dtype='object')
```

Why use `columns`?

Useful for:

- Seeing all column names
 - Renaming columns
 - Selecting columns dynamically
-

`df.index`

```
print(df.index)
```

Output:

```
RangeIndex(start=0, stop=5, step=1)
```

Why use `index`?

Index uniquely identifies rows.

Think of it like:

Row labels.

9. Accessing Columns and Rows

One of the MOST IMPORTANT Pandas concepts.

Access Single Column

```
print(df['CustomerName'])
```

Output:

```
0      John Smith
1      Sarah Lee
2      Ali Khan
```

Why Single Brackets?

Single bracket returns:

Series

Access Multiple Columns

```
print(df[['CustomerName', 'Price']])
```

Why Double Brackets?

Because:

```
[['A', 'B']]
```

means:

```
List of columns
```

and returns:

```
DataFrame
```

Access Particular Row using iloc

`iloc` means:

```
Integer-location indexing
```

Single Row with iloc

```
print(df.iloc[0])
```

Returns first row.

Multiple Rows using iloc

```
print(df.iloc[[0,2,4]])
```

Row and Column Selection with iloc

```
print(df.iloc[[0,1],[1,2]])
```

Meaning:

Rows 0,1 and columns 1,2

Why use iloc?

Useful when:

- Working with row positions
- Dataset indices are numeric
- Fast positional access needed

loc Indexing

`loc` means:

Label-based indexing

Example

```
print(df.loc[0])
```

loc with Condition

```
print(df.loc[df['Price'] > 100])
```

Difference Between loc and iloc

loc	iloc
Label based	Position based
Uses names	Uses integers
Includes stop label	Excludes stop position

10. Filtering Data

Filtering means selecting rows matching conditions.

One of the MOST USED operations in Pandas.

Basic Filtering

```
print(df[df['Price'] > 100])
```

Why Boolean Filtering Works?

Condition creates:

```
True / False mask
```

Pandas keeps rows where:

True

Equality Filter

```
print(df[df['Category'] == 'Electronics'])
```

Multiple Conditions

```
print(df[(df['Price'] > 100) & (df['Category'] == 'Electronics')])
```

Why Parentheses Needed?

Because:

```
&
```

is a bitwise operator.

Each condition must be grouped separately.

startswith()

```
print(df[df['CustomerName'].str.startswith('A')])
```

Why `.str`?

Because string methods in Pandas are accessed using:

```
.str
```

This enables vectorized string operations.

endswith()

```
print(df[df['Product'].str.endswith('p')])
```

isin()

```
print(df[df['Category'].isin(['Electronics', 'Furniture'])])
```

Why use `isin()`?

Useful when checking:

```
Multiple possible values
```

instead of many OR conditions.

NOT Condition

```
print(df[~df['Category'].isin(['Furniture'])])
```

Why `~`?

`~` means:

NOT

It reverses True/False values.

11. Updating Data

Updating values is extremely common in data cleaning.

Update Specific Value

```
df.loc[df['CustomerName'] == 'Sarah Lee', 'Price'] = 175
```

Why use loc for updates?

Because loc allows:

- Row selection
- Column selection
- Value assignment

all together.

Convert Text to Uppercase

```
df['Category'] = df['Category'].str.upper()
```

Output:

```
ELECTRONICS  
FURNITURE
```

Why `.str.upper()`?

Used for:

- Standardization
 - Cleaning inconsistent text
 - Data normalization
-

12. Deleting Data

Drop Row

```
df = df.drop(2, axis=0)
```

Why `axis=0`?

In Pandas:

```
axis=0 → rows  
axis=1 → columns
```

So:

```
axis=0
```

means delete row.

Drop Column

```
df = df.drop('Quantity', axis=1)
```

Delete using Condition

```
df = df[df['CustomerName'] != 'John Smith']
```

Why this works?

Because:

```
!=
```

creates boolean mask.

13. Cleaning Data

Cleaning data is one of the biggest tasks in real-world projects.

Check Missing Values

```
print(df.isnull().sum())
```

Why `isnull()`?

Returns:

```
True / False
```

for missing values.

Why `.sum()` after `isnull()`?

Because:

```
True = 1  
False = 0
```

So summing counts missing values.

Drop Missing Values

```
df.dropna(inplace=True)
```

Why `inplace=True`?

Means:

```
Modify original DataFrame directly
```

without creating copy.

Fill Missing Values

```
df.fillna({'Price': 0}, inplace=True)
```

Why fill missing values?

Machine learning models usually:

```
Cannot handle NaN values.
```

Rename Columns

```
df.rename(columns={'Price': 'ProductPrice'}, inplace=True)
```

Change Datatype

```
df['ProductPrice'] = df['ProductPrice'].astype(float)
```

Why `astype()`?

Used for:

- Memory optimization
- ML preprocessing
- Correct calculations

14. Data Analysis

Pandas is heavily used for analysis.

value_counts()

```
print(df['Category'].value_counts())
```

Output:

Electronics	3
Furniture	2

Why use `value_counts()`?

Useful for:

- Frequency analysis
 - Category analysis
 - Distribution checking
-

`sum()`

```
print(df['Quantity'].sum())
```

`mean()`

```
print(df['Price'].mean())
```

`groupby()`

```
print(df.groupby('Category')['Price'].mean())
```

Why `groupby()` is Powerful?

It enables:

```
SQL-like aggregations inside Python.
```

Very important for:

- Reporting
 - Analytics
 - Business intelligence
-

Sorting Data

Ascending Sort

```
print(df.sort_values(by='Price'))
```

Descending Sort

```
print(df.sort_values(by='Price', ascending=False))
```

Why Sorting Important?

Used for:

- Top products
 - Highest sales
 - Ranking
 - Reporting
-

15. Saving Data

After cleaning/analyzing data, we often save results.

Save CSV

```
df.to_csv('cleaned_data.csv', index=False)
```

Why `index=False`?

Otherwise Pandas saves row indices too.

Usually we don't want that.

Save Excel

```
df.to_excel('output.xlsx', index=False)
```

16. Real-World Workflow in Pandas

Typical workflow:

```
Load Data
→ Explore Data
→ Clean Data
→ Filter Data
→ Analyze Data
→ Save Results
```

17. Common Beginner Mistakes

Mistake 1

Using:

```
and
or
```

inside DataFrame filtering.

Wrong:

```
df[(df['Price'] > 100) and (df['Quantity'] > 1)]
```

Correct:

```
df[(df['Price'] > 100) & (df['Quantity'] > 1)]
```

Mistake 2

Forgetting parentheses in conditions.

Correct:

```
(df['Price'] > 100)
```

Mistake 3

Confusing loc and iloc.

Remember:

loc	iloc
Label	Position

Mistake 4

Modifying copy instead of original DataFrame.

Use:

```
inplace=True
```

when needed.

18. Important Interview Concepts

Difference Between Series and DataFrame

Series	DataFrame
1D	2D
Single column	Multiple columns
One datatype usually	Mixed datatypes

Why Pandas is Fast?

Because:

- Built on NumPy
- Vectorized operations
- Optimized C backend

Why Pandas is Used Everywhere?

Because it simplifies:

- Data cleaning
- Data manipulation
- Data analysis
- Reporting

19. Real-World Applications of Pandas

- Data Science
- Machine Learning
- Finance
- Business Analytics
- Data Engineering
- Reporting Systems
- ETL Pipelines
- Dashboard preparation

20. Pandas Quick Reference Cheat Sheet

File Loading

```
pd.read_csv()  
pd.read_excel()
```

Data Exploration

```
df.head()  
df.tail()  
df.info()  
df.describe()  
df.columns  
df.index
```

Selection

```
df['col']  
df[['col1', 'col2']]  
df.loc[]  
df.iloc[]
```

Filtering

```
==  
>  
<  
>=  
<=  
!=  
isin()
```

```
startswith()  
endswith()
```

Updating

```
loc[]  
str.upper()  
astype()  
rename()
```

Cleaning

```
isnull()  
dropna()  
fillna()
```

Analysis

```
value_counts()  
groupby()  
mean()  
sum()  
sort_values()
```

Saving

```
to_csv()  
to_excel()
```

Final Conclusion

Pandas is one of the MOST IMPORTANT Python libraries.

If you master Pandas:

You build strong foundations for:

- Data Science
- Machine Learning
- AI
- Analytics
- Data Engineering

Pandas + NumPy together form the core foundation of modern Python data analysis.

END OF NOTES